🎧 JamLite — Detailed Product Requirements Document (PRD) & System Design

---

🧠 1. PRODUCT OVERVIEW

Product Name: JamLite
Type: Real-time collaborative music platform
Core Idea: Public, discoverable Spotify Jam rooms with backend-controlled synchronization

JamLite is NOT a simple clone. It is a system that solves a real problem: 👉 Making music listening **social, real-time, and discoverable across strangers**.

🎯 2. PRODUCT VISION

JamLite transforms Spotify Jam from a private, invite-only feature into a **global public music network** where users:

- Discover live music rooms in real-time
- Join instantly without friction
- Experience perfectly synchronized playback
- Collaborate on queue building
- Connect through shared music taste

🔥 3. CORE DIFFERENTIATION

Problem in existing systems:

- Spotify Jam is closed and invite-only
- No discovery layer
- No social exploration

JamLite solves:

Capability	Description
Public Rooms	Anyone can discover and join rooms globally
Real-time Sync	Backend-controlled playback ensures no desync

Capability	Description
Discovery Layer	Browse rooms based on live activity
Taste Matching	Users find rooms based on genre/vibe

👉 This is a **social discovery + real-time sync system**, not just a UI project.

🧱 4. SYSTEM ARCHITECTURE (CRITICAL)

🔧 Architecture Model: Controller-Based

Unlike naive implementations, JamLite follows a **centralized control model**.

Data Flow:

Frontend → Backend → Spotify API → Redis → Clients

Explanation:

- Frontend NEVER directly controls Spotify
- Backend acts as **single source of truth**
- Redis stores real-time state
- All clients sync from backend state

Why this matters:

- Prevents playback drift
- Enables multi-user control
- Scales to many users per room

🔐 5. AUTHENTICATION SYSTEM

Spotify OAuth Flow

Functional Requirements:

- User clicks "Login with Spotify"
- Redirect to Spotify OAuth
- Backend exchanges code for tokens

Stored Data:

- Access token
- Refresh token
- Expiry time

Additional Data Pulled:

- User profile (name, avatar)
- Playback devices

Behavior:

- Token auto-refresh handled by backend
- User session persists across reloads

🌍 6. CORE FEATURE SET (DETAILED)

🧩 6.1 Public Room System

Description:

Rooms are the central entity of the system. Every room is public and discoverable.

Functional Requirements:

- Users can create a new room
- Rooms become instantly visible globally
- Rooms remain active while users are present

Room State Includes:

- Room ID
- Room name
- Host user
- Current playing track
- Playback state (playing/paused)
- Queue list
- Active users
- Metadata (genre, vibe)

Constraints:

- No private rooms in MVP
- One active playback per room

👥 6.2 Join / Leave System

Description:

Users can freely enter and exit any room.

Functional Behavior:

- Joining a room syncs user to current playback state
- Leaving removes user from active list
- UI updates reflect user count instantly

Edge Cases:

- If host leaves → assign new host OR fallback control logic
- If room empty → auto-delete or mark inactive

🎵 6.3 Playback Synchronization Engine

Description:

This is the most critical system.

Functional Requirements:

- Users can trigger play/pause/next
- Backend validates and executes action
- Spotify playback updated centrally

Sync Logic:

- Backend stores current timestamp
- Clients align playback based on server state

Guarantees:

- All users hear same track at same time
- No drift between clients

📃 6.4 Queue Management System

Description:

Shared queue where all users contribute.

Functional Requirements:

- Add song to queue
- Remove song (optional control rules)
- Maintain ordered list

Behavior:

- When song ends → next auto plays
- Queue updates propagate to all users

⚡ 6.5 Real-Time State Updates

Description:

Ensures UI reflects live state.

Data Synced:

- Current song
- Playback progress
- Queue
- Active users

Implementation Strategy:

- Phase 1: Polling (every few seconds)
- Phase 2: WebSockets (instant updates)

🌍 6.6 Discover Page (Core Entry Point)

Description:

Main application screen after login.

Functional Requirements:

- Display list of active public rooms
- Show:
- Room name
- Current track
- Number of users
- Allow joining room instantly

UI Behavior:

- Real-time updates of rooms
- Sorting (optional later)

✨ 7. ENHANCEMENT LAYER (GENRE + VIBE SYSTEM)

🎯 7.1 Room Metadata Extension

Rooms include taste descriptors:

- Genres (multiple allowed)
- Vibe (single descriptive tag)

Purpose:

- Enable meaningful discovery
- Improve user matching experience

🎛️ 7.2 Filtering System

Functional Requirements:

- Filter rooms by genre
- Filter rooms by vibe

Behavior:

- Filters dynamically update room list
- Multiple filters can be combined

🔍 7.3 Taste-Based Discovery

Description:

Users explore rooms based on musical preference.

Flow:

- Select genre/vibe → view matching rooms → join

⚡ Future Extension

- Automatic matching based on Spotify listening history
- Personalized recommendations

📱 8. USER FLOW (END-TO-END)

Step 1: Landing Page

- User learns about product
- Clicks login

Step 2: Login Page

- Spotify OAuth
- Redirect to Discover

Step 3: Discover Page

- Browse rooms
- Apply filters
- Join or create room

Step 4: Room Page

- Listen in sync
- Control playback
- Add songs

⚙️ 9. BACKEND SYSTEM DESIGN

🧠 Auth Service

Handles: - OAuth flow - Token refresh

🎵 Playback Controller

Handles: - Spotify API calls - Playback state management

🗒️ Room Service

Handles: - Room lifecycle - User membership

⚡ Real-Time Layer

Handles: - State sync - Event broadcasting

🗄️ Database Layer

Stores: - Rooms - Users - Metadata

🚀 10. DEVELOPMENT ROADMAP

Phase 1

- UI (Landing + Login)
- Routing
- Discover page (dummy data)

Phase 2

- Room creation
- Join/leave logic
- Queue system

Phase 3

- Backend playback control

Phase 4

- Redis integration
- Real-time sync

Phase 5

- Genre/vibe filters

⚠️ 11. NON-GOALS

Excluded from MVP:

- Chat system
- AI recommendation engine
- Voice features
- Complex analytics

💥 12. FINAL PRODUCT DEFINITION

JamLite is a:

- Real-time distributed system
- Public social music platform
- Backend-controlled synchronization engine
- Discovery-first product

🔒 FINAL NOTE

This PRD is the **official reference document**.

- Core features are fixed
- Architecture is locked

- Enhancements can be added without breaking system
